

RELATÓRIO TÉCNICO: COMPILADOR HOMI

Viviane Dilkin Endler

INTRODUÇÃO

O compilador Homi foi desenvolvido com o objetivo de abstrair a complexidade de arquivos de configuração YAML utilizados no ecossistema Home Assistant. Através de uma Linguagem de Domínio Específico (DSL) declarativa, permite que usuários leigos descrevam automações usando sentenças estruturadas e pseudo-naturais. Este documento apresenta a especificação técnica formal de todas as fases do compilador, conforme os requisitos estabelecidos na disciplina.

Implementação em Python e Arquitetura de Arquivos

Todo o projeto foi desenvolvido integralmente na linguagem Python (versão 3.x), sem dependência de geradores de código externos como Flex ou Bison. A arquitetura foi desenhada em módulos separados para garantir a aplicação correta dos conceitos de engenharia de compiladores:

- `main.py`: Orquestrador principal. Lê o arquivo `'.homi'`, instância as fases do compilador e gerencia a saída.
- `lexer.py`: Implementa o Analisador Léxico, varrendo o código com Expressões Regulares (simulando um DFA) para gerar os tokens.
- `sintatico.py`: Implementa o Analisador Sintático via Autômato de Pilha e Tabela Preditiva LL(1), montando a Árvore Sintática Abstrata (AST).
- `semantico.py`: Analisador Semântico que realiza a verificação de tipos (Type Checking) através de uma Tabela de Símbolos.
- `gerador.py`: O Back-End responsável por converter a AST validada na estrutura restrita e indentada do YAML do Home Assistant.

`Makefile`: Arquivo de automação auxiliar para execução de rotinas em ambientes Linux/macOS.

a. Descrição da GLC (identificando terminais e não-terminais)

A gramática livre de contexto (GLC) do compilador Homi foi projetada de forma estrita para aderir à classe LL(1), fatorada à esquerda e livre de ambiguidades, permitindo análise preditiva determinística (sem backtracking).

Conjunto de Símbolos:

Não-Terminais (VN): { Programa, ProgramaL, Automacao, Gatilho, OpGatilho, CondicaoOpt, OpCondicao, TempoOpt, AcoesL, Acao }

Terminais (VT): { QUANDO, ESTADO, OPERADOR, FACA, POR, FIM, E, SE, ENTITY_ID, STRING, TEMPO, COMANDO, ϵ }

Símbolo Inicial (S): Programa

Regras de Produção (P):

- (1) Programa → Automacao ProgramaL
- (2) ProgramaL → Automacao ProgramaL | ε
- (3) Automacao → QUANDO Gatilho CondicaoOpt AcoesL FIM
- (4) Gatilho → ENTITY_ID OpGatilho TempoOpt
- (5) OpGatilho → ESTADO STRING | OPERADOR STRING
- (6) CondicaoOpt → E SE ENTITY_ID OpCondicao | ε
- (7) OpCondicao → ESTADO STRING | OPERADOR STRING
- (8) TempoOpt → POR TEMPO | ε
- (9) AcoesL → FACA Acao AcoesL | ε
- (10) Acao → ENTITY_ID COMANDO

b. Descrição da especificação do Analisador Léxico

O Analisador Léxico tem a função de converter o fluxo de caracteres brutos em tokens classificados. Para a especificação, foi adotada a simulação de Autômatos Finitos Determinísticos (DFA) através do motor de Expressões Regulares (RegEx) do Python.

A tokenização ignora comentários (iniciados com #) e espaços em branco, mas intercepta quebras de linha para rastrear metadados. Abaixo está a tabela de especificação dos padrões léxicos mapeados pelo Scanner:

Token	RegEx	Exemplo
PALAVRA_CHAVE	<code>\b (QUANDO ESTADO FACA POR FIM E SE) \b</code>	QUANDO, ESTADO, FACA
TEMPO	<code>\b\d+(s min h)\b</code>	10s, 5min, 2h
OPERADOR	<code>(== != > = <= > <)</code>	>, <, ==
ENTITY_ID	<code>\b[a-z_]+\.[a-z0-9_]+\b</code>	light.corda_led_corredor
STRING	<code>"[^"]*"</code>	"on", "off", "30"
COMANDO	<code>\b (LIGAR DESLIGAR ALTERNAR) \b</code>	LIGAR, DESLIGAR

b.1. Tratamento de Metadados e Exemplo

Além da categorização, o Lexer monitora a ocorrência do caractere de quebra de linha (`\n`) para rastrear o índice da linha_atual, atrelando esse dado a cada token instanciado. Isso é vital para a precisão do subsistema de reporte de erros. Considere a linha "exemplo2_tempo.homi":

```
QUANDO binary_sensor.movimento ESTADO "off" POR 5min
```

O Analisador Léxico processa e emite:

```
[...]
{'tipo': 'PALAVRA_CHAVE', 'valor': 'QUANDO', 'linha': 1},
```

```
{'tipo': 'ENTITY_ID', 'valor': 'binary_sensor.movimento', 'linha': 1},
{'tipo': 'PALAVRA_CHAVE', 'valor': 'ESTADO', 'linha': 1},
{'tipo': 'STRING', 'valor': '"off"', 'linha': 1},
{'tipo': 'PALAVRA_CHAVE', 'valor': 'POR', 'linha': 1},
{'tipo': 'TEMPO', 'valor': '5min', 'linha': 1}
[...]
```

c. Descrição da especificação do Analisador Sintático

O Analisador Sintático valida se a fita de tokens respeita a hierarquia da GLC.

Autômato de Pilha e Tabela Preditiva:

A especificação exigiu rigor acadêmico, substituindo a Descida Recursiva por um Autômato de Pilha Orientado a Tabela. Uma matriz M cruza o Não-Terminal do topo da pilha com o Token atual (lookahead). O parser empilha as regras em ordem reversa e valida os tokens deterministicamente.

Ações Semânticas e Modo Pânico:

Marcadores semânticos (ex: @GATILHO_ID) foram injetados na Tabela M para construir a Árvore Sintática Abstrata (AST) durante o processo de derivação. Além disso, foi implementada a recuperação de erros via “Modo Pânico”: em caso de erro, o parser varre e descarta os tokens até encontrar a palavra-chave de sincronização "FIM", limpando o lixo da pilha e permitindo a continuidade da compilação das demais automações.

d. Descrição da especificação do Analisador Semântico

O Analisador Semântico opera sobre a AST gerada para garantir a coerência física e lógica da automação IoT.

Tabela de Símbolos Contextual

O componente central desta etapa é a Tabela de Símbolos, que atua como a memória formal de contexto do compilador. Como a DSL Homi não requer a declaração prévia de variáveis (visto que os identificadores já existem no ecossistema do Home Assistant), a tabela captura e mapeia dinamicamente cada `entity\_id` encontrado na primeira passagem. A tabela armazena o identificador literal, infere o seu tipo com base no prefixo do domínio e define o seu escopo (que no contexto de sistemas de automação residencial é sempre tratado como global).

Abaixo está a representação lógica da Tabela de Símbolos preenchida pelo compilador após a varredura dos scripts de teste:

Identificador (Nome da Entidade)	Tipo Inferido (Domínio)	Escopo Armazenado
binary_sensor.corredor_suite_luminance_motion_sensor_movimento	binary_sensor (Leitura)	global

alarm_control_panel.alarmo	alarm_control_panel (Leitura)	global
light.corda_led_corredor	light (Atuador/Escrita)	global
sensor.temperatura_quarto	sensor (Leitura Numérica)	global
climate.ar_condicionado_quarto	climate (Atuador Numérico)	global
fan.quarto	fan (Atuador/Escrita)	global

Especificação Two-Pass e Type Checking

A especificação adotou uma arquitetura de duas passagens (Two-Pass) apoiada por uma Tabela de Símbolos formal.

Passagem 1 (População): Varre a AST, extrai os identificadores fornecidos pelo usuário e infere seus domínios pelo prefixo (ex: "light.sala" é mapeado como tipo "light" e escopo global).

Passagem 2 (Type Checking): Avalia os comandos e operadores contra a Tabela de Símbolos. A regra de negócio proíbe, por exemplo, o envio de comandos de ação ("LIGAR") para domínios estritamente de leitura ("sensor"). Da mesma forma, proíbe a aplicação de operadores matemáticos relacionais (">", "<") em domínios puramente booleanos ("binary_sensor"). Em caso de violação, o compilador bloqueia a geração de código.

e. Exemplos de scripts Homi e os YAMLS resultantes

Abaixo são apresentados todos os casos de teste desenvolvidos, acompanhados de sua respectiva execução simulada no terminal (incluindo o gerador de código Back-End).

Exemplo 1: Automação base com condição (exemplo1_luz.homi)

```
QUANDO binary_sensor.corredor_suite_luminance_motion_sensor_movimento
ESTADO "on"
E SE alarm_control_panel.alarmo ESTADO "disarmed"
FACA light.corda_led_corredor LIGAR
FIM
```

```
PS C:\Users\Compiladores\trab> python main.py exemplo1_luz.homi
- id: '1722807263818'
  alias: Automacao Homi 1
  triggers:
  - trigger: state
    entity_id:
    - binary_sensor.corredor_suite_luminance_motion_sensor_movimento
    to: 'on'
  conditions:
  - condition: state
```

```

    entity_id: alarm_control_panel.alarmo
    state: 'disarmed'
actions:
- action: light.turn_on
  target:
    entity_id: light.corda_led_corredor
mode: single

```

Exemplo 2: Gatilho temporal e múltiplas ações (exemplo2_tempo.homi)

```

QUANDO binary_sensor.corredor_suite_luminance_motion_sensor_movimento
ESTADO "off" POR 5min
FACA light.corda_led_corredor DESLIGAR
FACA switch.tv_sala DESLIGAR
FIM

```

```

PS C:\Users\Compiladores\trab> python main.py exemplo2_tempo.homi
- id: '1722807263819'
  alias: Automacao Homi 1
  triggers:
  - trigger: state
    entity_id:
    - binary_sensor.corredor_suite_luminance_motion_sensor_movimento
    to: 'off'
    for:
      seconds: 300
  conditions:
  []
  actions:
  - action: light.turn_off
    target:
      entity_id: light.corda_led_corredor
  - action: switch.turn_off
    target:
      entity_id: switch.tv_sala
  mode: single

```

Exemplo 3: Operador relacional inteligente (exemplo3_ventilador.homi)

Observação: O gerador detecta o operador matemático ">" e transpila automaticamente de "trigger: state" para "trigger: numeric_state".

```

QUANDO sensor.temperatura_quarto > "30"
E SE climate.ar_condicionado_quarto ESTADO "off"
FACA fan.quarto LIGAR
FIM

```

```

PS C:\Users\Compiladores\trab> python main.py exemplo3_ventilador.homi
- id: '1722807263820'

```

```

alias: Automacao Homi 1
triggers:
- trigger: numeric_state
  entity_id:
  - sensor.temperatura_quarto
  above: 30
conditions:
- condition: state
  entity_id: climate.ar_condicionado_quarto
  state: 'off'
actions:
- action: fan.turn_on
  target:
    entity_id: fan.quarto
mode: single

```

Exemplo 4: Automação massiva sem condição (exemplo4_cinema.homi)

```

QUANDO input_boolean.modos_cinema ESTADO "on"
FACA light.sala_estar DESLIGAR
FACA media_player.tv_sala LIGAR
FACA light.fita_led_tv LIGAR
FIM

```

```
PS C:\Users\Compiladores\trab> python main.py exemplo4_cinema.homi
```

```

- id: '1722807263821'
alias: Automacao Homi 1
triggers:
- trigger: state
  entity_id:
  - input_boolean.modos_cinema
  to: 'on'
conditions:
[]
actions:
- action: light.turn_off
  target:
    entity_id: light.sala_estar
- action: media_player.turn_on
  target:
    entity_id: media_player.tv_sala
- action: light.turn_on
  target:
    entity_id: light.fita_led_tv
mode: single

```

Exemplo 5: Multidomínios (exemplo5_escova.homi)

```
QUANDO sensor.triumph_v2_90f2 ESTADO "charging"  
E SE binary_sensor.presenca_banheiro ESTADO "on"  
FACA light.espelho_banheiro LIGAR  
FACA switch.carregador_escova LIGAR  
FIM
```

```
PS C:\Users\Compiladores\trab> python main.py exemplo5_escova.homi
```

```
- id: '1722807263822'  
alias: Automacao Homi 1  
triggers:  
- trigger: state  
  entity_id:  
  - sensor.triumph_v2_90f2  
  to: 'charging'  
conditions:  
- condition: state  
  entity_id: binary_sensor.presenca_banheiro  
  state: 'on'  
actions:  
- action: light.turn_on  
  target:  
    entity_id: light.espelho_banheiro  
- action: switch.turn_on  
  target:  
    entity_id: switch.carregador_escova  
mode: single
```

Exemplo 6: Bloqueio Semântico de Atuação (exemplo6_erro.homi)

Tentativa de invocar um comando de ligar em um sensor passivo.

```
QUANDO binary_sensor.movimento ESTADO "on"  
FACA sensor.temperatura LIGAR  
FIM
```

```
PS C:\Users\Compiladores\trab> python main.py exemplo6_erro.homi
```

```
[ERRO SEMÂNTICO] Linha 2: O tipo 'sensor' é estritamente de leitura.  
Operação 'LIGAR' inválida em 'sensor.temperatura'.  
FALHA: Erros semânticos encontrados.
```

Exemplo 7: Bloqueio Semântico de Operador (exemplo7_erro_operador.homi)

Tentativa de aplicar operador matemático em um sensor binário (booleano).

```
QUANDO binary_sensor.presenca > "25"  
FACA light.espelho LIGAR  
FIM
```

```
PS C:\Users\Compiladores\trab> python main.py exemplo7_erro_operador.homi
[ERRO SEMÂNTICO] Linha 1: Operador relacional '>' não pode ser aplicado
à entidade 'binary_sensor.presenca' do tipo 'binary_sensor'.
FALHA: Erros semânticos encontrados.
```

f. Instruções de execução e compilação em múltiplos ambientes

O compilador foi desenvolvido em Python 3.x puro, dispensando a instalação de qualquer biblioteca ou dependência externa através de gerenciadores de pacotes (pip). Os comandos variam conforme o sistema operacional:

Instalação do Python:

Antes de executar os testes, é necessário ter a linguagem Python (versão 3.x) instalada na máquina.

- No Windows: Acesse o site oficial python.org, baixe o instalador mais recente e certifique-se de marcar a opção "Add Python to PATH" durante a instalação.
- No Linux: Geralmente já vem instalado. Caso contrário, utilize o comando `sudo apt install python3`.
- No macOS: Utilize o gerenciador de pacotes Homebrew executando `brew install python`.

Execução em Ambientes Windows (PowerShell / Prompt de Comando):

Como o Windows não possui interpretador nativo do comando "make", invoca-se diretamente o executável do Python, fornecendo o script do orquestrador ("main.py") seguido do arquivo contendo o código fonte Homi desejado:

```
PS C:\Users\Compiladores\trab> python main.py exemplo3_ventilador.homi
- id: '1722807263820'
  alias: Automacao Homi 1
  triggers:
  - trigger: numeric_state
    entity_id:
    - sensor.temperatura_quarto
    above: 30
  ...
```

Execução em Ambientes Linux / macOS (Unix-like Terminal):

Em sistemas Unix-like, utiliza-se o arquivo "Makefile" configurado na raiz para automatizar tarefas. O comando de execução utiliza a ferramenta Make e aponta para o cenário padrão:

```
PS C:\Users\Compiladores\trab> make run
Iniciando o compilador Homi com exemplo1_luz.homi...
- id: '1722807263818'
```



```
alias: Automacao Homi 1  
...
```

Caso queira remover os arquivos de cache e diretórios de otimização criados pelo interpretador Python (`\_\_pycache\_\_` e arquivos `.pyc`), execute na raiz:

```
PS C:\Users\Compiladores\trab> make clean  
Limpendo arquivos temporários do Python...
```

CONCLUSÃO

O compilador Homi demonstrou que as teorias clássicas de linguagens formais podem ser aplicadas de maneira contundente na simplificação de ecossistemas complexos de IoT. A orquestração das fases, utilizando uma varredura de DFA abstrata por expressões regulares, validação estrutural orientada estritamente por uma Tabela Preditiva LL(1) apoiada por pilha, controle de coerência lógica e de tipos contextuais via Tabela de Símbolos de duas passagens, e posterior geração de código YAML limpo, provou-se resiliente.